



Faculty of Engineering and Technology

Department of Computer Science

Artificial Intelligence COMP338

AI Based University Course Scheduling Using Genetic Algorithm

Course Project Report

Student Name	Narmeen Ibdah
Student ID	1230833
Section	LC #1
Instructor	AbedAlrahman Hamarsha

Abstract

This report presents the design, implementation, and evaluation of an AI-based university course scheduling system. The system uses a Genetic Algorithm implemented from scratch in Java to assign every course to a day, time, and room while satisfying mandatory academic constraints and reducing undesirable scheduling patterns. The implementation models courses, instructors, rooms, student groups, and lecture times as Java objects, evaluates complete timetables with a weighted penalty function, and improves candidate solutions using selection, crossover, mutation, elitism, and explicit termination conditions.

A JavaFX interface supports parameter configuration, final timetable visualization, fitness convergence plots, experimental comparisons, and comparison with Hill Climbing. The implemented system covers all minimum requirements and also supports Roulette Wheel Selection, one-point and two-point crossover, random and swap mutation, multiple dataset sizes, and Genetic Algorithm versus Hill Climbing comparison. Representative runs produced complete schedules with zero hard violations for every tested dataset from 10 to 50 courses, while runtime increased and fitness decreased as the problem size grew.

Keywords

University course scheduling, Genetic Algorithm, constraint optimization, Java, JavaFX, Hill Climbing, timetable generation.

Figure of Contents

Figure 1:Eclipse project structure.....	9
Figure 2Main JavaFX interface and parameter controls.....	12
Figure 3Genetic Algorithm experimental results	16
Figure 4Final generated timetable	17
Figure 5Generation versus best fitness.....	18
Figure 6Genetic Algorithm versus Hill Climbing	19

Introduction

1.1 Background

University timetabling is a constrained optimization problem in which a collection of courses must be assigned to limited rooms and lecture periods. A valid schedule must avoid simultaneous use of the same instructor, room, or student group, and it must respect room capacity, room type, and instructor availability. In addition to validity, a useful timetable should avoid undesirable periods and reduce long gaps between the lectures of the same student group.

1. The search space grows rapidly because each course can be assigned to many combinations of days, times, and rooms. For five days, four periods per day, and six rooms, one course has 120 possible positions. Even a ten-course dataset therefore produces an extremely large number of possible complete timetables, making exhaustive enumeration impractical.

1.2 Motivation for an Artificial Intelligence Approach

A Genetic Algorithm is suitable because it searches a large solution space without evaluating every possible timetable. It maintains a population of complete schedules, rewards schedules with fewer violations, and combines and modifies promising solutions over successive generations. The method can handle several hard and soft constraints in one numerical fitness function and can continue improving a valid solution after conflicts have been eliminated.

1.3 Project Objectives

- Generate a complete timetable automatically for datasets containing 10, 20, 30, 40, or 50 courses.
- Eliminate instructor, room, and student-group conflicts whenever possible.
- Respect room capacity, room type, and instructor availability.
- Reduce early classes, late classes, and unnecessary student-group gaps.
- Allow the user to configure population size, generation limit, crossover rate, mutation rate, and genetic operators.
- Display the final timetable, quality measures, convergence graph, experimental results, and algorithm comparison in JavaFX.

1. Problem Formulation

2.1 Scheduling Entities

Entity	Main attributes	Role in the schedule
Course	ID, name, students, instructor, type, student group	The activity that must be scheduled exactly once
Instructor	ID, name, unavailable lecture times	Teaches assigned courses and cannot overlap
Room	ID, capacity, type	Hosts a course when capacity and type are suitable
Student Group	Group ID	Courses for the same group cannot overlap
Lecture Time	Day and time	Represents one academic period
Gene	Course, room, lecture time	Represents one complete course assignment
Chromosome	List of genes	Represents one complete timetable

2.2 Time and Room Domain

The scheduling week contains Sunday, Monday, Tuesday, Wednesday, and Thursday. Each day provides four one-hour periods starting at 08:00, 09:00, 10:00, and 11:00. UniversityData supports datasets of 10, 20, 30, 40, or 50 courses and defines 10 instructors, five student groups, four lecture rooms, and two laboratories.

Room	Capacity	Type
R101	50	Lecture
R102	40	Lecture
R103	35	Lecture
R104	25	Lecture
LAB1	30	Lab
LAB2	25	Lab

2.3 Hard Constraints

Hard constraints define timetable validity and receive a high penalty of 100 for each violation.

ID	Constraint	Violation condition
H1	Instructor conflict	Two courses taught by the same instructor use the same lecture time.
H2	Room conflict	Two courses use the same room at the same lecture time.
H3	Student-group conflict	Two courses belonging to the same student group overlap.
H4	Room capacity	The course enrollment is greater than the selected room capacity.
H5	Room type	A laboratory course is assigned to an ordinary lecture room, or the required type does not match.
H6	Instructor availability	A course is placed in a period when its instructor is unavailable.

2.4 Soft Constraints

Soft constraints improve timetable convenience and receive a smaller penalty of 10 for each violation.

ID	Constraint	Evaluation
S1	Avoid early classes	Add one soft violation when a class starts at 08:00.
S2	Avoid late classes	Add one soft violation when a class starts at 11:00.
S3	Minimize student gaps	Penalize nonconsecutive lectures for the same group on the same day.

Optimization Goal

The primary goal is to minimize hard violations. Once a valid timetable is found, the algorithm continues to minimize soft violations. The different penalty weights ensure that one hard violation is considered much worse than several soft violations.

3 Genetic Algorithm

3.1 Chromosome and Gene Representation

A chromosome is a complete timetable. Its gene list has the same size and order as the course list, so each course appears exactly once. A gene stores references to one Course, one Room, and one LectureTime. This direct object representation avoids decoding numeric slot identifiers and makes constraint evaluation readable.

Gene index	Course	Day	Time	Room
0	Artificial Intelligence	Monday	10:00	R102
1	Database	Sunday	09:00	R101
2	Computer Graphics	Tuesday	11:00	LAB1

The example above illustrates the representation only. The generated values change from one run to another because initialization, selection, crossover, and mutation use random choices.

3.2 Initial Population

PopulationGenerator creates the requested number of chromosomes. For every course, it randomly chooses one room and one lecture time from UniversityData. Random initialization provides diversity, which allows the Genetic Algorithm to explore different areas of the search space. Invalid initial schedules are allowed because the fitness function guides the population toward feasibility.

3.3 Fitness Function

FitnessCalculator counts hard and soft violations, calculates the total penalty, and converts the penalty into a fitness value that must be maximized.

$$Penalty = WH_H + WS_S$$

$$Penalty = 100H + 10S$$

$$Fitness = \frac{1}{1 + Penalty}$$

With this formulation, fitness is always positive and increases as the penalty decreases. A conflict-free timetable with three soft violations has penalty 30 and fitness $1/31 = 0.03226$. A perfect schedule would have fitness 1.0.

3.4 Parent Selection

3.4.1 Tournament Selection

Tournament Selection randomly samples a small group of chromosomes and returns the chromosome with the highest fitness. It is simple, efficient, and provides controllable selection pressure. It is the default selection method.

3.4.2 Roulette Wheel Selection

Roulette Wheel Selection assigns each chromosome a probability proportional to its fitness. Better chromosomes are more likely to be selected, but weaker chromosomes can still contribute genetic material. This optional operator was implemented to compare selection strategies.

3.5 Crossover

3.5.1 Two Point Crossover

Two cut positions are selected. The assignments between the two cuts are copied from the second parent, while the remaining assignments are copied from the first parent. Because the course order is fixed, crossover changes only room and lecture-time assignments and never removes a course.

3.5.2 One Point Crossover

A single cut position divides both parents. The child receives the first segment from one parent and the remaining segment from the other. This operator creates a simpler exchange pattern and is available for comparison with two-point crossover.

3.6 Mutation

3.6.1 Random Reassignment Mutation

For each gene, mutation occurs according to the configured probability. A mutated gene receives a newly selected room and lecture time. This operator introduces new assignments that may not exist in either parent.

3.6.2 Swap Mutation

Swap Mutation selects two course genes and exchanges their room and lecture-time assignments. It preserves the set of currently used positions while exploring a different course-to-position mapping.

3.7 Elitism and Termination

Elitism copies the best chromosome or a small elite fraction directly into the next generation. This protects the best timetable from being lost through random crossover or mutation. The search terminates when an acceptable solution is found, the maximum number of generations is reached, or the configured stagnation condition is met. The implementation stores the best fitness from each generation for the convergence chart.

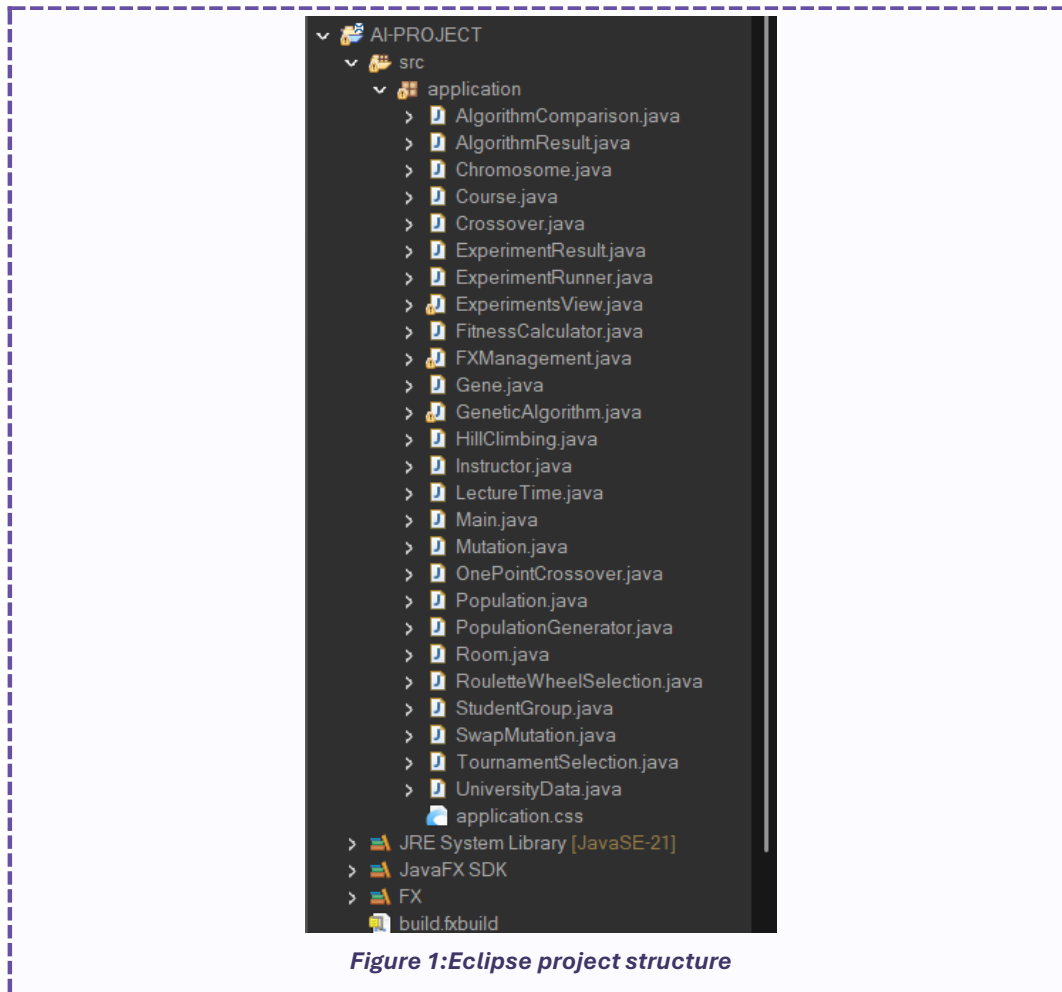
3.8 Overall Procedure

1. Generate a random initial population.
2. Evaluate every chromosome using FitnessCalculator.
3. Sort or identify chromosomes according to fitness.
4. Copy elite chromosomes to the new population.
5. Select parents using Tournament or Roulette Wheel Selection.
6. Apply one-point or two-point crossover according to the crossover probability.
7. Apply random reassignment or swap mutation according to the mutation probability.
8. Evaluate the new population and record the best fitness.
9. Repeat until a termination condition is satisfied, then return the best timetable.

4 Code Explanation

4.1 Technologies and Program Organization

The project is implemented in Java and developed as a JavaFX project in Eclipse. All project classes are organized in the application package. The Genetic Algorithm components were implemented directly rather than calling a ready-made optimization library. Java collections are used for the population and schedule data, while JavaFX observable collections connect results to tables and charts.



4.2 Main Classes and Responsibilities

Class	Responsibility
Main	Starts the JavaFX application and delegates interface construction to FXManagement.
FXManagement	Builds the interface, reads parameters, runs algorithms, and updates tables, metrics, and charts.
UniversityData	Creates courses, rooms, lecture times, instructors, student groups, and expanded datasets.
Course	Stores course ID, name, enrollment, instructor, required type, and student group.
Instructor	Stores instructor identity and the lecture times in which the instructor is unavailable.
Room	Stores room ID, capacity, and lecture or laboratory type.
LectureTime	Stores day and time; equals and hashCode compare these two values.
StudentGroup	Identifies students who attend the same set of required courses.
Gene	Stores one course assignment as Course plus Room plus LectureTime.
Chromosome	Stores a complete timetable and its hard, soft, penalty, and fitness values.
Population	Maintains the current collection of candidate chromosomes.
PopulationGenerator	Creates randomized chromosomes for the initial population.
FitnessCalculator	Counts constraint violations and calculates penalty and fitness.
TournamentSelection	Selects the best chromosome from a random tournament.
RouletteWheelSelection	Selects chromosomes probabilistically according to fitness.
Crossover	Implements two-point crossover.
OnePointCrossover	Implements one-point crossover.

Class	Responsibility
Mutation	Implements random room and lecture-time reassignment.
SwapMutation	Swaps the assignments of two genes.
GeneticAlgorithm	Coordinates initialization, evaluation, operators, elitism, history, and termination.
HillClimbing	Implements a local-search baseline for algorithm comparison.
ExperimentRunner	Executes predefined parameter and dataset-size experiments.
ExperimentResult	Stores one experimental configuration and its measured results.
ExperimentsView	Displays experiment controls and results in JavaFX.
AlgorithmComparison	Runs GA and Hill Climbing on comparable data and returns both result histories.
AlgorithmResult	Stores algorithm name, quality measures, iterations, runtime, and convergence history.

4.3 Important Data Structures

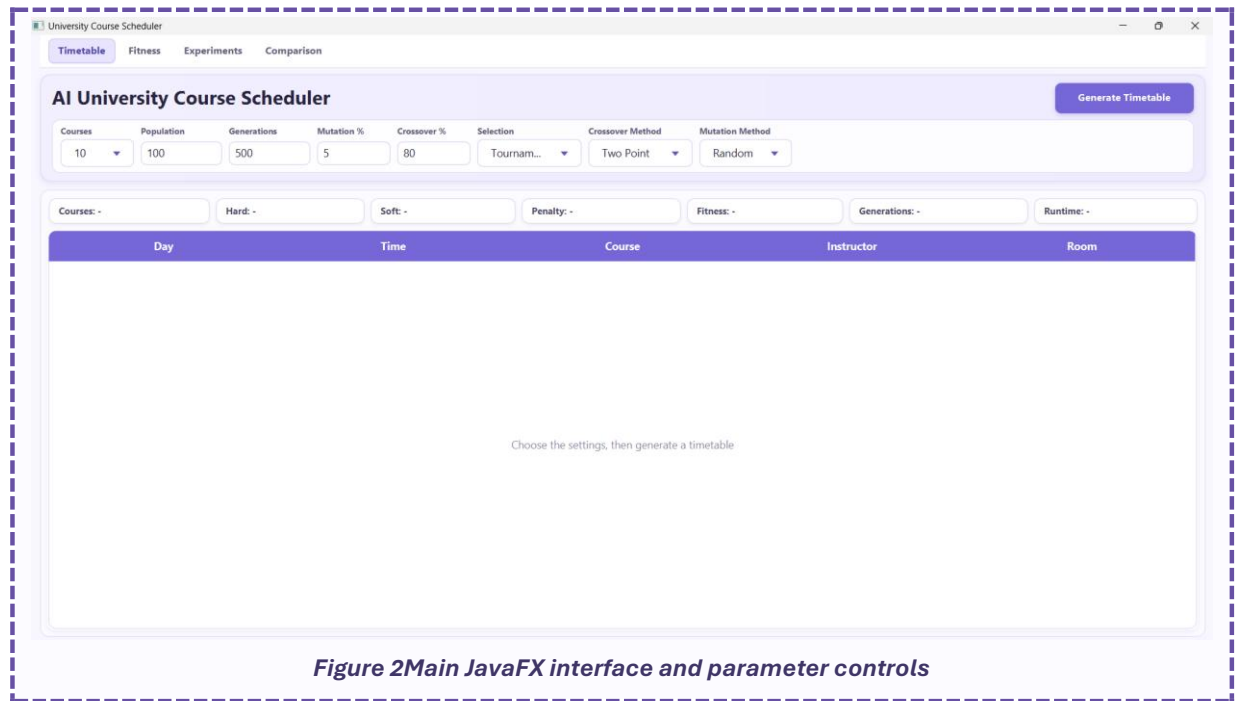
Structure	Stored data	Reason for use
ArrayList<Gene>	Course assignments in a chromosome	Maintains ordered access for crossover and mutation.
ArrayList<Chromosome>	Candidate timetables	Supports evaluation, sorting, selection, and replacement.
ArrayList<LectureTime>	Available and unavailable periods	Supports random selection and availability checks.
ObservableList	Rows shown by JavaFX tables	Automatically connects result collections to TableView.
XYChart.Series	Fitness history by generation or iteration	Provides convergence visualization in LineChart.

4.4 Constraint Evaluation

FitnessCalculator first checks each gene independently for room capacity, room type, and instructor availability. It then compares pairs of genes that share the same lecture time to detect instructor, room, and student-group conflicts. Finally, it evaluates early periods, late periods, and gaps for each student group. The calculated values are stored in the chromosome so the interface and experiments can display them without recomputing the schedule.

4.5 JavaFX User Interface

FXManagement separates the interface from Main. The top navigation contains Timetable, Fitness, Experiments, and Comparison pages. The settings area accepts course count, population size, generation limit, mutation percentage, crossover percentage, selection method, crossover method, and mutation method. The Generate Timetable button validates the inputs, builds the selected dataset, runs the algorithm, and updates every relevant visual component.



4.6 Input Validation and Usability

Numeric values are parsed from text fields and checked before execution. Invalid population sizes, generation limits, or probabilities produce an alert instead of starting the algorithm. The interface uses a light purple stylesheet, consistent spacing, readable table headers, result cards, and responsive content pages.

5 Experimental Work

5.1 Experimental Method

Experiments were conducted through ExperimentRunner and displayed in ExperimentsView. In each comparison, one parameter was varied while the other main parameters were held near the default configuration. The recorded measures were best fitness, number of hard violations, number of soft violations, number of generations, and runtime in milliseconds. Because the algorithms are stochastic, repeated runs can produce different values even with the same configuration.

Parameter	Default value
Courses	10
Population size	100
Maximum generations	500
Mutation rate	5%
Crossover rate	80%
Elitism	Enabled
Selection	Tournament
Crossover	Two Point
Mutation	Random

5.2 Experiment 1 Population Size

Population sizes 20, 50, 100, and 200 were tested. A larger population examines more candidate timetables in every generation and may improve diversity, but it also increases the work performed per generation.

Courses	Population	Mutation	Crossover	Generations	Best fitness	Hard	Soft	Runtime ms
10	20	5%	80%	12	0.02439	0	4	2
10	50	5%	80%	4	0.02439	0	4	3
10	100	5%	80%	4	0.02439	0	4	3
10	200	5%	80%	5	0.03226	0	3	8

All four population sizes produced feasible timetables with zero hard violations. Population 200 achieved the best fitness, 0.03226, by reducing soft violations from four to three, but its runtime increased to 8 ms. This shows that a larger population can improve solution quality while requiring more computation.

5.3 Experiment 2 Mutation Rate

Courses	Population	Mutation	Crossover	Generations	Best fitness	Hard	Soft	Runtime ms
10	100	1%	80%	5	0.02439	0	4	4
10	100	5%	80%	4	0.02439	0	4	3
10	100	10%	80%	6	0.02439	0	4	4
10	100	20%	80%	5	0.02439	0	4	4

Mutation rates of 1%, 5%, 10%, and 20% reached the same representative fitness of 0.02439 with zero hard violations and four soft violations. Their generation counts and runtimes differed slightly. Because mutation is random, repeated runs would be needed before declaring one rate consistently superior; very low mutation may reduce exploration, while very high mutation may disrupt useful assignments.

5.4 Experiment 3 Crossover Rate

Courses	Population	Mutation	Crossover	Generations	Best fitness	Hard	Soft	Runtime ms
10	100	5%	60%	6	0.03226	0	3	3
10	100	5%	80%	4	0.02439	0	4	3
10	100	5%	90%	4	0.03226	0	3	2

In this representative run, crossover rates of 60% and 90% reached fitness 0.03226 with three soft violations, while 80% reached fitness 0.02439 with four soft violations. The difference is small and stochastic, so 80% remains a reasonable default rather than a guaranteed optimum.

5.5 Experiment 4 Dataset Size

Courses	Population	Mutation	Crossover	Generations	Best fitness	Hard	Soft	Runtime ms
10	100	5%	80%	4	0.02439	0	4	3
20	100	5%	80%	12	0.02439	0	4	12
30	100	5%	80%	89	0.02439	0	4	103
40	100	5%	80%	193	0.01639	0	6	261
50	100	5%	80%	296	0.00662	0	15	554

Increasing the dataset from 10 to 50 courses increased both generations and runtime. Every tested size reached zero hard violations. Runtime rose from 3 ms for 10 courses to 554 ms for 50 courses, while the 50-course case retained 15 soft violations and fitness 0.00662. The larger cases are therefore feasible but harder to optimize using the same rooms, periods, and parameter settings.

5.6 Advanced Operator Experiments

Tournament and Roulette Wheel selection, one-point and two-point crossover, and random and swap mutation were compared using 20 courses, population 100, 500 maximum generations, 80% crossover, and 5% mutation. Only the operator under investigation was changed. The measured single-run results are summarized below.

Comparison	Option A	Option B	Better fitness	Lower runtime	Observation
Selection	Tournament	Roulette Wheel	Tie: 0.02439	Roulette: 35 ms	Both had 0 hard and 4 soft violations; Tournament used 17 generations and Roulette used 39.
Crossover	One Point	Two Point	Tie: 0.02439	One Point: 30 ms	Both had 0 hard and 4 soft violations; Two Point used fewer generations (17 vs. 23).
Mutation	Random	Swap	Random: 0.02439	Random: 50 ms	Random had 4 soft violations; Swap had 8 and fitness 0.01235.

(a) Population, mutation, and crossover experiments

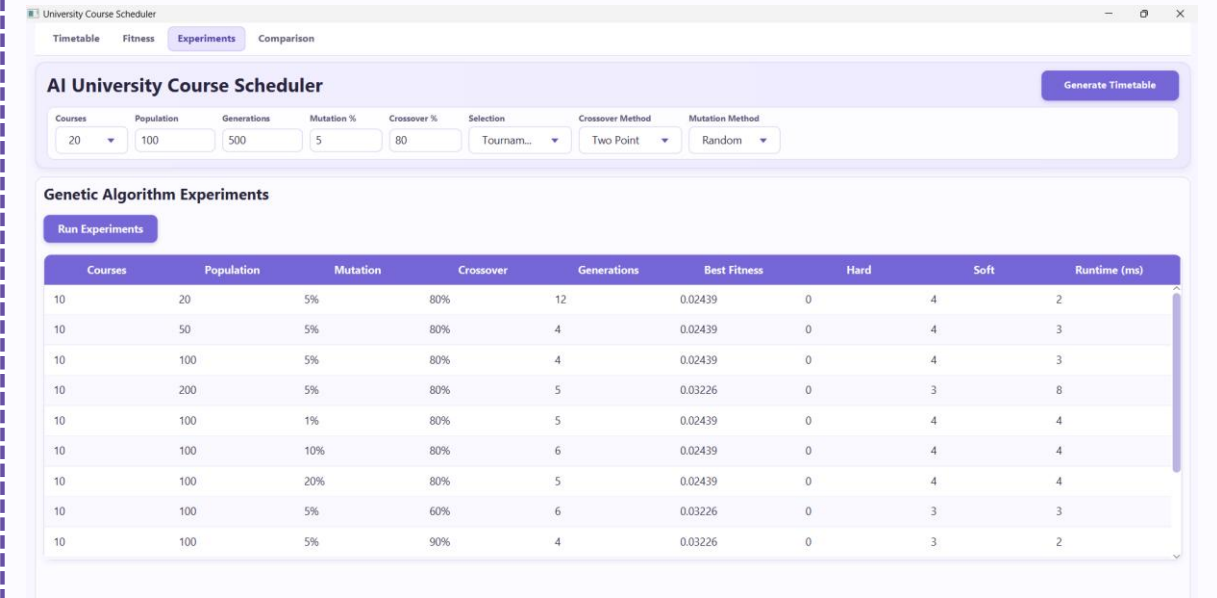
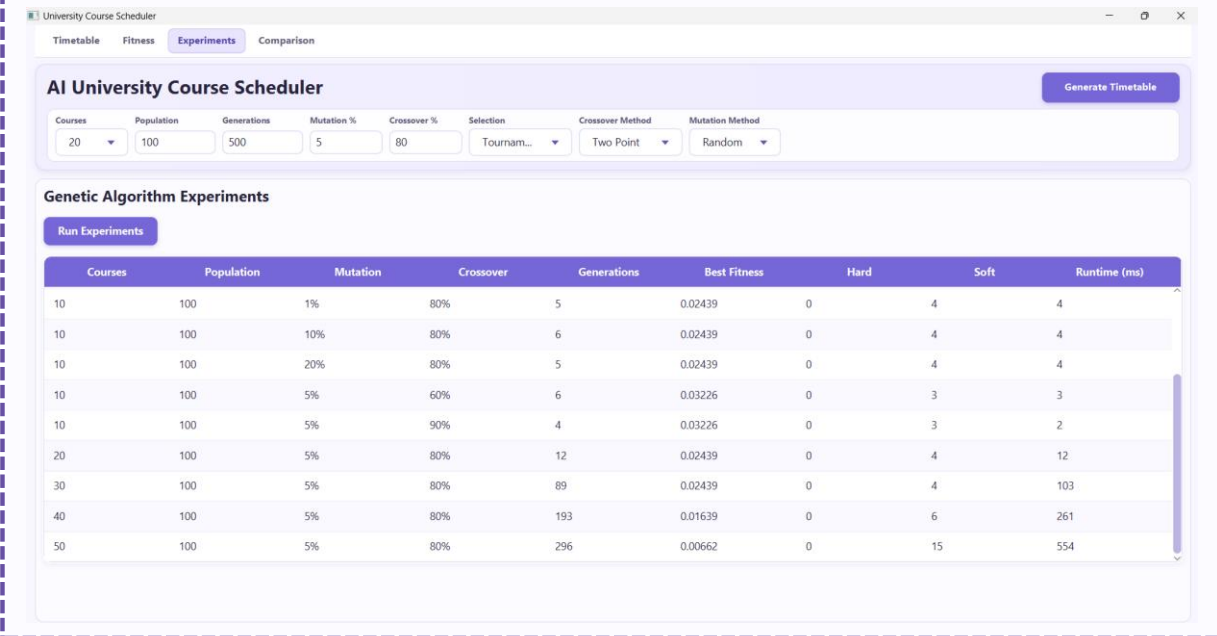


Figure 3 Genetic Algorithm experimental results

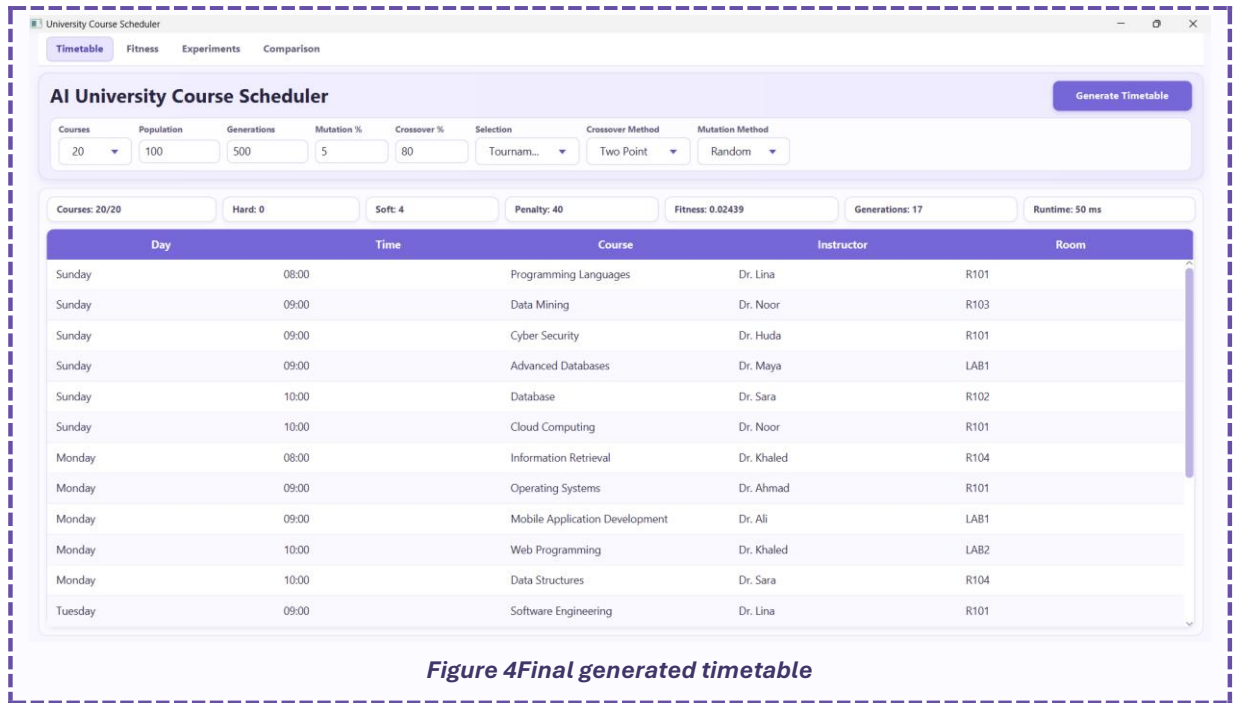
(b) Dataset-size experiment



6 Results

6.1 Final Timetable

The final solution is sorted by day and time and displayed in a five-column JavaFX table containing Day, Time, Course, Instructor, and Room. Using the preferred configuration, all 20 courses were scheduled with zero hard violations and four soft violations.



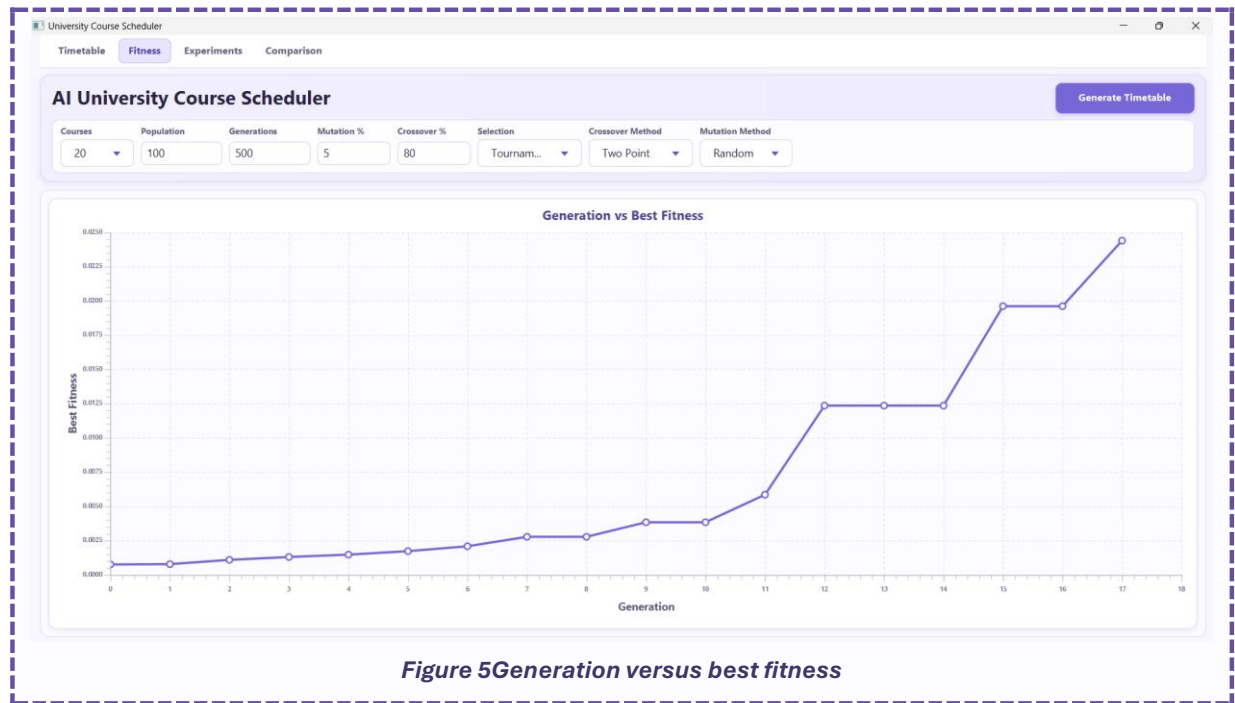
6.2 Final Quality Measures

Measure	Final value
Courses scheduled	20/20
Hard violations	0
Soft violations	4
Final penalty	40
Best fitness	0.02439
Generations	17
Runtime	50 ms

The preferred timetable scheduled all 20 requested courses. A hard-violation value of zero confirms that all mandatory constraints were satisfied. The four soft violations represent convenience preferences rather than invalid assignments. With penalty 40, the reported fitness is $1 / (1 + 40) = 0.02439$.

6.3 Fitness Convergence

The Generation versus Best Fitness graph records the best solution in each generation. A rising curve demonstrates that selection and genetic operators are discovering schedules with lower penalties. Flat regions indicate generations in which no better schedule was found, while a later increase indicates that crossover or mutation created an improved assignment. Elitism prevents the plotted best fitness from decreasing between generations.

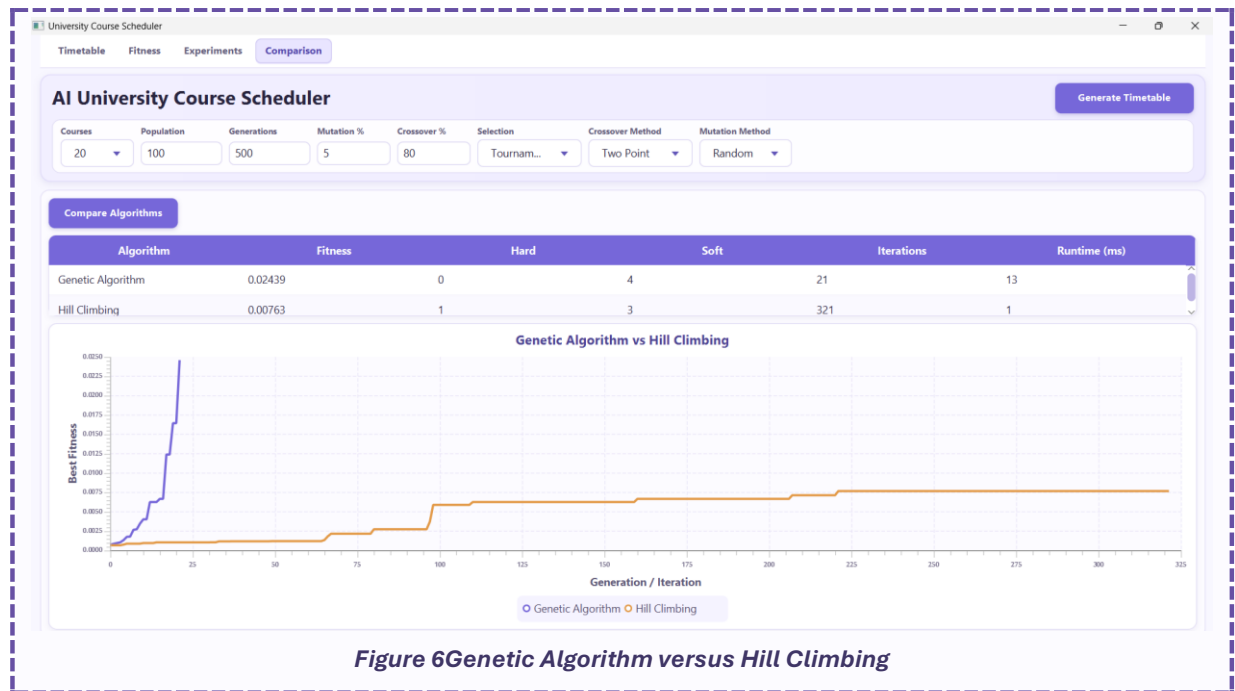


6.4 Genetic Algorithm and Hill Climbing Comparison

Hill Climbing starts from one timetable and repeatedly accepts an improved neighboring timetable. It is fast because it explores a local neighborhood, but it can stop at a local optimum. The Genetic Algorithm explores several solutions simultaneously and uses crossover and mutation to search beyond one local neighborhood, usually at the cost of additional runtime.

Algorithm	Fitness	Hard	Soft	Iterations	Runtime ms
Genetic Algorithm	0.02439	0	4	21	13
Hill Climbing	0.00763	1	3	321	1

The Genetic Algorithm produced a feasible schedule with zero hard violations, four soft violations, and fitness 0.02439. Hill Climbing completed in only 1 ms but stopped with one hard violation and fitness 0.00763. This representative run illustrates the expected tradeoff: local search was faster, while the population-based search produced the valid and higher-quality timetable.



6.5 Discussion of Results

The experiments show that feasibility and convenience must be evaluated separately. A schedule with higher numerical fitness can still be unacceptable if it contains a hard violation; therefore, hard violations are inspected explicitly in every table. For small datasets, several configurations quickly generated valid schedules. For larger datasets, runtime and the number of generations increased, and the 50-course representative run became more difficult under the same fixed resources and algorithm settings.

The results also demonstrate that parameter effects are not strictly monotonic. Increasing population, crossover, or mutation does not guarantee improvement. The operators must balance exploitation of good schedules with exploration of new assignments. A practical configuration for this implementation is population 100, maximum 500 generations, crossover 80%, mutation 5%, Tournament Selection, Two Point Crossover, and Random Mutation, followed by additional operator comparisons.

6.6 Limitations

- The experiment values depend on random choices; a single run is not sufficient for statistical conclusions.
- The current soft constraints cover early classes, late classes, and student gaps, but do not yet include every possible institutional preference.
- Expanded datasets reuse the available days, times, rooms, and scheduling patterns, so the larger cases are intentionally more constrained.
- Runtime measurements depend on the computer, Java version, and concurrent system activity.

7 Conclusion

The project successfully implemented an AI-based university course scheduling system using a Genetic Algorithm written from scratch in Java. The timetable representation assigns every course to a room and lecture time, and the fitness function evaluates six hard constraints and three soft constraints with different weights. The algorithm includes population initialization, selection, crossover, mutation, elitism, termination, and fitness-history tracking.

The JavaFX application makes the system practical to test and demonstrate. Users can change dataset size and Genetic Algorithm parameters, choose alternative genetic operators, generate a readable timetable, inspect quality metrics, view convergence, run controlled experiments, and compare the Genetic Algorithm with Hill Climbing. Representative results confirm that the implementation can generate complete schedules with zero hard violations for multiple dataset sizes and that parameter choices influence both fitness and runtime.

Future improvements could add repeated-run averages, standard deviations, CSV export, automatic report export, richer instructor preferences, limits on student working days, adaptive mutation, repair operators, and hybrid search that applies Hill Climbing to the best Genetic Algorithm chromosomes.

References

1. COMP338 Artificial Intelligence Course Project. AI Based University Course Scheduling Using Genetic Algorithm. Faculty of Engineering and Technology, Department of Computer Science, Summer Semester 2025/2026.
2. Oracle. Java Platform and JavaFX documentation. Used for Java language collections, JavaFX controls, TableView, and LineChart concepts.
3. Project source code. Java implementation classes in the application package, developed for the COMP338 course project.

